

# **SIMATS ENGINEERING**

## **CO4 - ASSESSMENT TOOL 1**

### **Design and Develop Modal**

**COURSE NAME: Deep Learning**

**COURSE CODE: MLA0408**

#### **COURSE OUTCOME COVERED**

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems.. (BL-6)

**Assessment Tool Weightage: 30%**

#### **QUESTIONS:**

**Total Marks:25**

#### **Q1. Transfer Learning**

Design a transfer learning framework for a plant disease detection system using a pre-trained CNN model. Justify the choice of layers to freeze and fine-tune, and propose suitable data augmentation techniques to improve classification performance.

#### **Q2. DenseNet and PixelNet**

Develop a CNN-based architecture by integrating DenseNet and PixelNet concepts for semantic image segmentation in autonomous driving. Explain how dense connectivity and pixel-wise prediction improve the overall segmentation accuracy.

#### **Q3. Bidirectional RNN and Encoder–Decoder**

Create an encoder–decoder sequence-to-sequence model using Bidirectional RNNs for multilingual machine translation. Illustrate the architecture and explain how bidirectional context enhances translation quality.

#### **Q4. BPTT and LSTM**

Design an LSTM-based network to predict stock market trends from historical time-series data. Explain how Backpropagation Through Time (BPTT) is applied during training and justify why LSTM is preferred over a standard RNN.

#### **Q5. Integrated Deep Learning Model**

Develop a deep learning solution for automatic video caption generation by integrating transfer learning for feature extraction with an LSTM-based encoder–decoder architecture. Justify the role of each component and explain how the complete system generates meaningful captions.

### Code of Conduct:

**I G. Naveen Kumar Reddy (192525288)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

### RUBRICS FOR CALCULATION:

| Criteria                | Weightage | Level 4 – Exemplary                                                                       | Level 3 – Proficient                                      | Level 2 – Developing                          | Level 1 – Beginning                              |
|-------------------------|-----------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------|-----------------------------------------------|--------------------------------------------------|
| Problem Understanding   | 20%       | Clearly understands problem constraints, edge cases, and competitive requirements         | Understands problem with minor gaps in constraints        | Partial understanding; misses key constraints | Misinterprets problem or constraints             |
| Algorithm               | 25%       | Selects optimal algorithm with best time and space complexity for competitive constraints | Uses efficient algorithm with minor scope for improvement | Uses basic/inefficient approach               | No clear algorithmic approach                    |
| Code Implementation     | 20%       | Writes correct, efficient, and clean code that passes all constraints                     | Code works with minor errors or inefficiencies            | Partially correct code; fails for some cases  | Code does not execute or gives incorrect results |
| Competitive Performance | 20%       | Solves problem within time limits and handles large inputs effectively                    | Solves within acceptable time with minor delays           | Struggles with time limits or larger inputs   | Unable to solve within time constraints          |
| Test Case Handling      | 15%       | Handles all test cases including edge and hidden cases                                    | Handles most test cases                                   | Misses important edge cases                   | Fails on multiple test cases                     |

# ANSWERS

## Q1. Transfer Learning Framework for Plant Disease Detection

### 1. Problem Understanding

The goal is to classify plant leaf images into healthy or diseased categories (and further into specific disease classes) using a Convolutional Neural Network (CNN). Since labelled agricultural datasets (e.g., PlantVillage) are relatively small compared to large-scale datasets like ImageNet, training a deep CNN from scratch would overfit. Transfer learning solves this by reusing knowledge already learned by a network trained on millions of general images.

### 2. Proposed Framework / Architecture

- Base model: Use a pre-trained CNN such as ResNet50, VGG16, or EfficientNetB0 trained on ImageNet.
- Input pipeline: Resize leaf images to the base network's expected input size (e.g.,  $224 \times 224 \times 3$ ), normalize pixel values using the same mean/std used during pre-training.
- Feature extractor: Remove the original classification head (fully connected + softmax layer for 1000 ImageNet classes).
- New classifier head: Add Global Average Pooling  $\rightarrow$  Dense(256, ReLU)  $\rightarrow$  Dropout(0.5)  $\rightarrow$  Dense(N classes, Softmax), where N is the number of disease categories.
- Two-phase training: Phase 1 — train only the new head with the base frozen; Phase 2 — unfreeze the top few convolutional blocks and fine-tune with a very low learning rate.

### 3. Which Layers to Freeze and Which to Fine-tune (Justification)

- Freeze early/shallow layers: The first few convolutional blocks learn generic, low-level features such as edges, colours, textures and simple shapes. These are common to almost all natural images, including leaves, so they do not need retraining and freezing them prevents overfitting and saves computation.
- Fine-tune deeper/later layers: The last 2–3 convolutional blocks learn dataset-specific, high-level features (object parts, complex patterns). Disease symptoms such as lesion shapes, discoloration patterns and spot textures are domain-specific, so these layers must be fine-tuned so the network adapts from generic 'ImageNet objects' to 'leaf disease patterns'.
- Replace and fully train the classification head: Since the number of output classes is completely different from ImageNet, the head is randomly initialised and trained completely from scratch.
- Use a small learning rate (e.g.,  $1e-5$ ) during fine-tuning of unfrozen layers to avoid destroying the useful pre-trained weights, while the new head can use a higher rate (e.g.,  $1e-3$ ).

### 4. Data Augmentation Techniques Proposed

- Geometric transformations: random rotation ( $\pm 30^\circ$ ), horizontal/vertical flip, random zoom and translation — leaves can appear in any orientation in the field.
- Colour/lighting augmentation: brightness, contrast and saturation jitter — to simulate different lighting/weather conditions during image capture.

- Random cropping and shearing — to make the model robust to partial leaf visibility and camera angle.
- Gaussian noise / blur addition — simulates camera focus issues and improves robustness to real-world image quality.
- Cutout / random erasing — forces the network to rely on multiple disease regions rather than a single spot, improving generalisation.
- Class balancing via augmentation — apply heavier augmentation to under-represented disease classes to counter class imbalance, which is common in agricultural datasets.

### ***5. Expected Benefit***

This transfer-learning framework converges faster, needs far fewer labelled images, and achieves higher classification accuracy than training a CNN from scratch, while the augmentation strategy improves robustness and reduces overfitting on real-world field images.

## **Q2. DenseNet + PixelNet based Architecture for Semantic Segmentation (Autonomous Driving)**

### ***1. Problem Understanding***

Autonomous driving requires pixel-level understanding of the scene (road, pedestrians, vehicles, lanes, signs) — this is semantic segmentation, where every pixel must be assigned a class label. The architecture must combine strong feature reuse (DenseNet) with precise pixel-wise prediction (PixelNet).

### ***2. Proposed Architecture***

- Encoder (Feature Extraction) – DenseNet backbone: Use Dense Blocks where each layer receives feature maps from all preceding layers as input (via concatenation, not addition). This is followed by Transition Layers ( $1 \times 1$  conv + pooling) that control feature-map growth and downsample spatial resolution.
- Decoder / Prediction Head – PixelNet style: Instead of predicting from only the final feature map, hypercolumn features are extracted — features from multiple intermediate DenseNet layers (shallow, mid, deep) are upsampled to the input resolution and concatenated for every pixel.
- A pixel-wise MLP ( $1 \times 1$  convolutions) is applied on the concatenated multi-scale hypercolumn to classify each pixel into one of the driving-scene classes (road, vehicle, pedestrian, sky, sign, etc.).
- Skip connections between encoder and decoder preserve fine spatial boundaries (important for detecting thin objects like poles, lane markings).
- Final layer: Softmax over classes per pixel, followed by a CRF or bilinear upsampling refinement for smooth boundaries.

### ***3. How Dense Connectivity Improves Accuracy***

- Feature reuse: Every layer has direct access to the gradients and features of all previous layers, so useful low-level (edges, lane lines) and high-level (object shape) features are never lost, unlike plain CNNs.

- Strong gradient flow: Direct connections mitigate the vanishing-gradient problem, allowing much deeper networks to be trained effectively — important since driving scenes require deep semantic understanding.
- Parameter efficiency: Because features are reused rather than relearned, DenseNet achieves high accuracy with fewer parameters than ResNet/VGG of similar depth, which matters for the real-time constraints of autonomous vehicles.
- Implicit deep supervision: Since every layer is connected to the loss through short paths, the network is implicitly regularised, reducing overfitting.

#### **4. How Pixel-wise Prediction Improves Accuracy**

- Multi-scale context: By combining hypercolumn features from shallow (fine boundary detail) and deep (semantic context) layers, PixelNet-style prediction captures both 'what' the object is and 'where' its precise boundary lies.
- Fine-grained boundaries: Standard CNN classifiers only use the final coarse feature map, which loses spatial precision. Pixel-level prediction using multi-layer features restores sharp segmentation boundaries — critical for safely distinguishing a pedestrian's edge from the road.
- Per-pixel loss (cross-entropy per pixel) directly optimises the segmentation task rather than an image-level classification proxy, giving denser supervision signal per image.

#### **5. Overall Benefit for Autonomous Driving**

The DenseNet encoder provides rich, gradient-stable, reusable features while the PixelNet-style multi-scale pixel prediction head restores spatial precision — together producing accurate, sharp-boundaried, real-time-capable semantic segmentation maps needed for safe navigation.

## **Q3. Encoder–Decoder Seq2Seq Model with Bidirectional RNN for Multilingual Machine Translation**

### **1. Problem Understanding**

Machine translation converts a variable-length source-language sentence into a variable-length target-language sentence. This is a sequence-to-sequence (seq2seq) problem, naturally solved with an encoder–decoder architecture, where a Bidirectional RNN (BiRNN) encoder captures full sentence context in both directions.

### **2. Architecture**

- Input embedding layer: Each source word/subword token is converted to a dense vector (word embeddings, optionally multilingual embeddings such as fastText/mBERT-style).
- Encoder — Bidirectional RNN (BiLSTM/BiGRU): A forward RNN reads the sentence left-to-right ( $h \rightarrow_1 \dots h \rightarrow_n$ ) and a backward RNN reads it right-to-left ( $h \leftarrow_n \dots h \leftarrow_1$ ). At each time step  $t$ , the two hidden states are concatenated:  $h_t = [h \rightarrow_t ; h \leftarrow_t]$ , forming a context-rich representation for every word.
- Context/attention layer: Instead of compressing the whole sentence into one fixed vector, an attention mechanism computes a weighted combination of all encoder hidden states at every decoding step, letting the decoder 'look back' at the most relevant source words.

- Decoder — unidirectional RNN (LSTM/GRU): Generates the target sentence one token at a time. At each step it takes the previous output word, its own hidden state, and the attention context vector to predict the next word via a softmax over the target vocabulary.
- Training: Teacher forcing (feeding the true previous target word during training) with cross-entropy loss; at inference, beam search is used to find the most probable target sentence.
- Multilingual extension: A shared encoder/decoder with a language-id token prepended to the input sentence (as in Google's multilingual NMT) allows one model to translate between many language pairs.

### ***3. How Bidirectional Context Enhances Translation Quality***

- Full sentence context per word: A unidirectional RNN encodes a word using only the words before it; a BiRNN encodes each word using both preceding and following words, which is essential because word meaning and grammatical role (e.g., resolving pronoun references, disambiguating homographs) often depend on later words in the sentence.
- Better handling of word order differences: Languages differ in syntax (e.g., subject-verb-object vs. subject-object-verb). Bidirectional context helps the encoder build representations that are less order-biased, making it easier for attention to align source and target words correctly regardless of language structure.
- Improved long-range dependency capture: Combined with attention, BiRNN hidden states let the decoder attend to any part of the sentence (before or after the current focus point), improving translation of long or complex sentences.
- Reduced information bottleneck: Because every source word has a rich, context-aware representation, the model does not need to compress the entire sentence meaning into a single vector, reducing translation errors especially in long sentences.

### ***4. Expected Outcome***

This BiRNN encoder–decoder with attention produces higher BLEU scores than plain unidirectional seq2seq models, handles ambiguous words and long sentences better, and generalises well across multiple language pairs when trained multilingually.

## **Q4. LSTM-based Network for Stock Market Trend Prediction (with BPTT)**

### ***1. Problem Understanding***

Stock price movement is a sequential, time-series problem with long-term temporal dependencies (trends, momentum, seasonality) and noisy short-term fluctuations. This requires a model that can remember relevant patterns over long time windows — LSTM is well suited for this.

### ***2. Proposed Architecture***

- Input: A sliding window of the past  $T$  time steps (e.g., 60 days) of features — Open, High, Low, Close, Volume, and technical indicators (moving averages, RSI) — normalised (e.g., MinMax scaling).
- Stacked LSTM layers: LSTM(128 units, return\_sequences=True) → Dropout(0.2) → LSTM(64 units) → Dropout(0.2) — stacking allows the network to learn hierarchical temporal patterns (short-term fluctuations in the first layer, longer trend patterns in the second).

- Dense output layer: Dense(1) for regression (predicting next-day price/return) or Dense(3, softmax) for classification (up/down/neutral trend).
- Loss function: Mean Squared Error for regression or categorical cross-entropy for trend classification; optimiser: Adam.

### ***3. How BPTT is Applied During Training***

- The LSTM is unrolled across all  $T$  time steps of the input window, creating a computational graph equivalent to a very deep feedforward network (one 'layer' per time step).
- Forward pass: The input sequence is passed through the unrolled network, computing hidden states  $h_t$  and cell states  $c_t$  at each time step, and finally the prediction and loss at the last (or every) time step.
- Backward pass: The error gradient is propagated backward through time, from the final time step to the first, using the chain rule — gradients with respect to weights are accumulated across all time steps since the same weight matrices are shared/reused at every step.
- Weight update: Once gradients from all time steps are summed, a single weight update is applied (via Adam/SGD) — this is what distinguishes BPTT from ordinary backpropagation: gradients flow through the temporal/recurrent connections, not just through network depth.
- Truncated BPTT: For long sequences, gradients are propagated back only a fixed number of steps (e.g., 60) rather than the entire history, to keep training computationally feasible.

### ***4. Why LSTM is Preferred over a Standard RNN (Justification)***

- Vanishing/exploding gradient problem: In a standard RNN, repeated multiplication of the same weight matrix during BPTT causes gradients to shrink (vanish) or grow (explode) exponentially over long sequences, so the network 'forgets' information from many time steps ago — a serious limitation for stock data where trends unfold over weeks/months.
- Gated memory cell: LSTM introduces a cell state  $c_t$  regulated by three gates — forget gate (decides what old information to discard), input gate (decides what new information to store), and output gate (decides what to output) — allowing gradients to flow through the cell state largely unchanged (an approximately additive, constant-error-carousel path), which prevents vanishing gradients.
- Long-term dependency capture: Because of the gating mechanism, LSTM can selectively retain relevant long-term trend information (e.g., a multi-week upward momentum) while still being sensitive to recent short-term price shocks — something a plain RNN cannot do reliably.
- Empirically, LSTMs consistently outperform vanilla RNNs on noisy, long-sequence financial time-series in both accuracy and training stability.

### ***5. Expected Outcome***

The stacked LSTM trained with BPTT captures both short-term volatility and long-term trend signals in stock data, producing more stable and accurate trend/price predictions than a standard RNN.



## Q5. Integrated Deep Learning Model for Automatic Video Caption Generation

### 1. Problem Understanding

Video captioning requires (a) understanding visual content across frames (spatial + temporal features) and (b) generating a grammatically correct, semantically relevant natural-language sentence. This naturally combines Transfer Learning (for visual feature extraction) with an LSTM-based Encoder–Decoder (for sequence generation) — a CNN-LSTM / 'S2VT'-style architecture.

### 2. Proposed Integrated Architecture

- Component A — Transfer Learning Feature Extractor (CNN): A pre-trained CNN (e.g., InceptionV3 or ResNet50, trained on ImageNet) is used to extract a fixed-length feature vector from each sampled video frame, using the network up to the last pooling layer (classification head removed). Frames are sampled at fixed intervals (e.g., 1 frame per second) to reduce redundancy.
- Optional 3D-CNN branch (e.g., pre-trained C3D/I3D) can be added in parallel to capture motion/temporal action features across consecutive frames, concatenated with the 2D-CNN spatial features.
- Component B — Encoder LSTM: The sequence of per-frame CNN feature vectors is fed into an encoder LSTM (or BiLSTM) time-step by time-step, producing a final hidden state / context vector that summarises the entire video's visual content and temporal evolution.
- Attention mechanism: At each decoding step, attention weights are computed over all encoder time-step outputs so the decoder can focus on the most relevant frames when generating each word (e.g., focusing on frames showing the actual action when generating a verb).
- Component C — Decoder LSTM (language model): Generates the caption word-by-word, taking the previous word embedding, its hidden state, and the attention context vector at each step, producing a softmax distribution over the vocabulary. A <start> token begins generation and an <end> token terminates it.
- Training: Cross-entropy loss with teacher forcing; inference uses beam search for the most fluent, high-probability caption.

### 3. Justification of Each Component's Role

- Transfer learning (CNN): Provides high-quality, semantically rich visual features without needing millions of labelled video frames — since video-captioning datasets are much smaller than ImageNet, training a CNN from scratch would badly overfit; the pre-trained CNN already 'knows' how to recognise objects, scenes and actions from general images.
- Encoder LSTM: Compresses the variable-length sequence of frame features into a temporally-aware representation, capturing how the scene changes/evolves over time (e.g., a person picking up an object vs. putting it down), which a single-frame CNN alone cannot capture.
- Attention: Prevents information loss from compressing an entire video into one fixed vector, letting the decoder dynamically reference the most relevant moments for each generated word — improving caption relevance and detail.



- **Decoder LSTM:** Acts as a conditional language model, ensuring the generated caption is not just visually grounded but also fluent and grammatically valid English (or the target language), by learning word-to-word sequential dependencies from caption training data.

#### ***4. How the Complete System Generates Meaningful Captions***

Given an input video, sampled frames are passed through the pre-trained CNN to obtain per-frame semantic features. These are encoded by the LSTM encoder into a temporal summary of the video. During generation, the attention-enabled LSTM decoder produces the caption word-by-word, at each step attending to the most relevant frames and using the language model's learned grammar to choose the next most probable word, continuing until the <end> token is produced. Because visual understanding (transfer learning) is decoupled from language generation (LSTM decoder) but jointly trained end-to-end, the resulting captions are both visually accurate and linguistically fluent — e.g., correctly describing 'a man is riding a bicycle on the road' rather than only recognising isolated objects like 'man', 'bicycle', 'road'.

#### ***5. Expected Benefit***

This integration leverages transfer learning to solve the data-scarcity problem in visual feature learning while the LSTM encoder–decoder with attention solves the temporal modelling and language-generation problem — together producing accurate, fluent, and contextually meaningful automatic video captions.